

UI Authorization Architecture Study

Analysis & Implementation Strategy for Permission-Based Browser Authorization

Target Domain: Category UI
System: Central Store Application

Date: August 10, 2026
Scope: UI Authorization Architecture

1. Executive Summary

This study evaluates the existing authorization architecture in the Central Store application to design a permission-based authorization mechanism for browser UI routes, beginning with the **Category Master** module. The objective is to achieve strict access control while maximizing code reuse and preserving the current REST API behavior.

2. Analysis of Existing Authentication & Authorization Architecture

2.1 REST API Authentication (Bearer Token)

The REST API uses `require_permission(permission_code)` defined in `app/dependencies/permissions.py`. It obtains the authenticated user via `get_current_user` (in `app/dependencies/auth.py`), which extracts the Bearer token from the Authorization header using FastAPI's `HTTPBearer()`. If unauthenticated, it raises a JSON 401 Unauthorized response.

2.2 Browser UI Authentication (Cookie-Based)

The Browser UI uses `get_current_user_ui()` defined in `app/dependencies/ui_auth.py`. It extracts the JWT access token from the browser's `HttpOnly` cookie (`request.cookies.get('access_token')`). If the cookie is missing, expired, or invalid, it raises an `HTTPException(status_code=303, headers={'Location': '/login'})`, redirecting the user to the login page.

3. Core Permission Engine & Code Reusability

The core permission evaluation is performed by `has_permission()` in `app/services/permission_service.py`:

```
def has_permission(db: Session, user_id: int, permission_code: str) -> bool:
    permission_code = permission_code.strip().upper()
    permissions = get_user_permissions(db=db, user_id=user_id)
    return any(permission.code.upper() == permission_code for permission in permissions)
```

Reusability Finding: `has_permission()` is completely decoupled from HTTP protocols, request parsing, and response formatting. It depends only on `db: Session`, `user_id: int`, and `permission_code: str`. Therefore, **100% of the permission evaluation logic can be reused for UI authorization** without any code duplication.

4. Smallest Architectural Change Required

To introduce UI-compatible permission enforcement, the minimal architectural addition is creating a new dependency factory `require_permission_ui(permission_code: str)` inside `app/dependencies/ui_auth.py`:

```
def require_permission_ui(permission_code: str):
    def permission_checker_ui(
        current_user: User = Depends(get_current_user_ui),
        db: Session = Depends(get_db),
    ) -> User:
        if not has_permission(db=db, user_id=current_user.id, permission_code=permission_code):
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail='You do not have permission to access this page.'
            )
        return current_user
    return permission_checker_ui
```

Key aspects of this architecture:

- **Authentication Reuse:** Delegates user authentication to `get_current_user_ui` (maintains cookie checking + 303 redirect to `/login`).
- **Logic Reuse:** Delegates permission checking to `has_permission()` in `permission_service.py`.
- **UI Error Handling:** Returns a 403 Forbidden response (or custom access denied view) when an authenticated user lacks the specific permission.

5. Proposed Dependency Flow for Category UI Routes

Below is the complete route-to-permission mapping proposed for `app/routers/ui/category.py`:

UI Route Handler	HTTP Method & Path	Required Permission	Dependency Execution Flow
<code>list_categories</code>	GET <code>/category/</code>	<code>CATEGORY_VIEW</code>	<code>get_current_user_ui</code> → <code>has_permission(CATEGORY_VIEW)</code>
<code>category_table</code>	GET <code>/category/table</code>	<code>CATEGORY_VIEW</code>	<code>get_current_user_ui</code> → <code>has_permission(CATEGORY_VIEW)</code>
<code>new_category</code>	GET <code>/category/new</code>	<code>CATEGORY_CREATE</code>	<code>get_current_user_ui</code> → <code>has_permission(CATEGORY_CREATE)</code>
<code>save_category</code>	POST <code>/category/new</code>	<code>CATEGORY_CREATE</code>	<code>get_current_user_ui</code> → <code>has_permission(CATEGORY_CREATE)</code>
<code>edit_category</code>	GET <code>/category/{id}/edit</code>	<code>CATEGORY_UPDATE</code>	<code>get_current_user_ui</code> → <code>has_permission(CATEGORY_UPDATE)</code>
<code>update_category_ui</code>	POST <code>/category/{id}/edit</code>	<code>CATEGORY_UPDATE</code>	<code>get_current_user_ui</code> → <code>has_permission(CATEGORY_UPDATE)</code>
<code>delete_category_ui</code>	POST <code>/category/{id}/delete</code>	<code>CATEGORY_DELETE</code>	<code>get_current_user_ui</code> → <code>has_permission(CATEGORY_DELETE)</code>

6. Implementation Recommendations & Next Steps

1. Add `require_permission_ui()` to `app/dependencies/ui_auth.py`.
2. Apply `Depends(require_permission_ui(...))` to route functions in `app/routers/ui/category.py`.
3. Optionally render a clean 403 HTML page for browser users when permission check fails.
4. Verify browser navigation, HTMX live search calls, and form submissions under various user roles.